LFTC: Lab 4 - Documentation

> ***Statement**: For the domain specific programming language from Lab 1, use FA (Finite Automata) and RG (Regular Grammars) for identifiers and constants*

Finite Automata

Each automaton stores states and transition rules that describe the valid form of an identifier or an integer constant.

- States = what we have read until now: (in our DSL: q0 nothing **or** q1, 0 (final state), or q2 a digit from 1-9 or a 0 but coming after a nonzero digit)
- Transition = going from one state to the other. For exp, q0 → q1 means we didn't read anythin yet → oop we read a digit

We defined 2 FA's, one for identifiers and one for constants. The automata are defined in `automaton.c`

Mapping from formal notation to variables

```
//M = (Q, Σ, δ, q₀, F) is an automaton
//Q = stateCount (3 = {0,1,2})
//Σ = allowedCharacters[]
//δ = transitions[]
//q0 = initialState
//F = the set of states i where isFinalState[i] = 1 (true)
```

Identifier Automaton

```
//IDENTIFIERS AUTOMATON
//-- States--:
// 1. q0 = no characters have been read, initial state, not final
// 2. q1 = a char has been read, not initial, final
//--Transitions--:
// 1. q0 -[a-zA-Z] - q1
// 2. q1 -[a-zA-Z0-9_] - q1
```

Integer Constant Automaton

```
//CONSTANTS AUTOMATON
//-- States--:
// 1. q0 = no characters have been read, initial state, not final
// 2. q1 = a 0 has been read. not initial, final
// 3. q2 = a nonzero first digit has been, followed by zero or more digits, not initial,
final
//--Transitions--:
// 1. q0 -[0]- q1 ( from nothing, we read a 0)
// 2. q0 -[1-9] - q2 ( from nothing, we read a [1-9] digit)
// 3. q2 -[0-9] - q2 (from a 1-9 digit we read a [0-9] digit)
```

Conversion from FA to RG

```
//Seminar Notation:
// G = (N, Σ, P, S) a RG ( Regular Grammar)
// N = the set of nonterminals (ex. {S,A}
// Σ = terminals' alphabet (ex. {a,b})
// P = the productions' set (ex. S → aA, A → b)
// S = the start symbol ( an element of N ) (ex. S)
```

```
//--- Conversion from AF -> RG ---
// Q (stateCount) = N
// Σ (allowedCharacters[]) = Σ
// q0 (initialState) = S
// transition q0 —a→ q2 = production q0 → a q2
// q0 —a→ q1, where q1 final = q0 → a q1 | a

// RG specific: If the initial state is final, also add S → ε: the empty string is accept
ed.
  // but in our FA's our initial states are not final, so we do not need an ε production.
```